CHATTSS: Improving Test Suite Simplification via Large Language Models

Gaolei Yi[1], Yuan Zhao[2,*], Runkang Feng[1], Quanjun Zhang[3], and Zhenyu Chen[4,*]
[1]State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China
[2]Laboratory of Data Intelligence and Interdisciplinary Innovation, Nanjing University, Nanjing, China
[3]School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing, China
[4]Shenzhen Research Institute of Nanjing University, Shenzhen, China
yigaolei@smail.nju.edu.cn, allenzcrazy@gmail.com, runkangfeng@smail.nju.edu.cn,
quanjunzhang@njust.edu.cn, zychen@nju.edu.cn
*corresponding author

2025 25th International Conference on Software Quality, Reliability and Security (QRS) | 978-1-6654-7771-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/QRS65678.2025.00021

Abstract—As a critical component of software testing activities, regression testing plays an indispensable role in ensuring the correctness of software systems after changes. With the increasing scale and complexity of modern software, a pressing challenge arises: how to efficiently select the most effective test cases from existing test suites for regression testing, thereby reducing the associated cost.

Although numerous methods have been proposed for test suite reduction, most of them rely on the assumption that test cases are independent of each other. In this paper, we present CHATTSS, a novel test case simplification approach powered by LLM. Unlike conventional test suite reduction that only shrinks the size of the test suite without altering individual test cases, CHATTSSleverages the program analysis capabilities of LLM to decompose test cases into fine-grained test atoms. It then applies appropriate reduction algorithms to perform more precise and effective test suite simplification.

We conducted experiments on seven open-source projects, comprising over 10,000 test cases, to evaluate the effectiveness of CHATTSS. Experimental results demonstrate that CHATTSS exhibits strong simplification performance across multiple evaluation dimensions, confirming its potential as an efficient and scalable TSR solution.

Keywords–Regression Test; Test Suite Reduction; Test Suite Simplification; LLM

1. INTRODUCTION

As a crucial software testing activity aimed at ensuring the stable and normal operation of existing functions after software changes, regression testing plays an extremely important role throughout the software life cycle[1]. Regression testing not only verifies newly implemented features but also conducts a comprehensive re-examination of existing functionalities. This ensures seamless integration between new and existing components, thereby enhancing overall software quality and reinforcing user trust. During regression testing, test cases are primarily derived from executed test suite T. Consequently, the simplest regression testing strategy is to re-execute all test cases on the modified version of the system under test (SUT) to ensure comprehensive coverage of all functional aspects [2][3]. However, as the software system scales and its complexity increases, this approach becomes progressively inadequate for managing the risks introduced by frequent code changes. To address this challenge, researchers have proposed test suite reduction (TSR), a technique that significantly decreases the size and execution time of regression testing by identifying and retaining only those test cases that have the greatest impact on software quality.

TSR is typically regarded as an optimization problem targeting reduction requirements[4][5][6][7]. Based on whether the simplified test suite can satisfy all the original suite's requirements, it is categorized into adequate TSR and inadequate TSR [8]. In earlier related research, single-objective optimization algorithms are predominantly used for TSR, such as those based on statement coverage [6][9][10][11]. The most widely applied algorithm in single-objective TSR is the greedy algorithm [8][12]. As research complexity increases, more intricate reduction requirements, such as fault detection rate [10][13][14] and execution cost [4][15], are being adopted. Consequently, multi-objective optimization algorithms, such as NSGA-II, are now applied to TSR, as proposed by Yoo et al [6]. Additionally, there are studies that employ hybrid algorithms to address varying TSR scenarios [16][17]. However, the majority of the current research still focuses on single-objective optimization [18]. Furthermore, to the best of our knowledge, despite the existence of test case order dependencies within test suites in TSR tasks [14][19][20][21], most of the current research on TSR assumes no inherent dependencies between test cases or test orders within the suite. Complex dependencies within test cases may negatively impact the effectiveness of TSR.

TSR primarily target unit test cases, aiming to reduce regression testing costs by minimizing the size of the test suite. However, in practical applications, these test cases often exhibit implicit dependencies, which pose significant challenges to effective reduction. If such dependencies among unit test cases can be decoupled, the subsequent reduction process may further minimize the test suite size and improve regression testing efficiency. Leveraging the powerful natural language understanding and code analysis capabilities of LLM, we propose CHATTSS. To distinguish it from conventional TSR techniques, we define it as a test suite simplification method. CHATTSS decomposes test cases into fine-grained, indivisible units, referred to as "test atoms", and employs multi-dimensional reduction objectives along with a multi-

2693-9177/25/$31.00 ©2025 IEEE
DOI 10.1109/QRS65678.2025.00021

objective optimization algorithm to minimize the number of test cases while preserving overall test effectiveness.

We collected over 10,000 test cases from seven diverse projects to conduct a comprehensive evaluation of CHATTSS across multiple dimensions, including code coverage, mutation score, test suite size, and execution cost. The study first validates CHATTSS's capabilities in program analysis and test case repair by examining metrics such as changes in test suite size, fluctuations in the number of test cases, and the pass rate of test atoms. Second, a comparative analysis with traditional class- and method-level test suite reduction demonstrates that the test atom mechanism can significantly enhance simplification efficiency while minimizing requirement loss. Finally, cross-project evaluations of different algorithms are conducted to inform adaptive algorithm selection strategies. Experimental results show that CHATTSS delivers superior performance in terms of test decomposition granularity, simplification efficiency, and overall test suite quality preservation.

In summary, the contributions of this paper are as follows:

- To the best of our knowledge, this is the first work to apply LLM to the task of test suite simplification. By leveraging the program analysis capabilities of LLM, we significantly enhance the efficiency and precision of test suite simplification.
- We propose CHATTSS, a novel LLM-based test suite simplification approach that decomposes test cases into fine-grained test atoms, enabling more accurate and effective test suite simplification.
- We evaluate CHATTSS on over 10,000 test cases across seven real-world projects. The results demonstrate that CHATTSS is both efficient and effective in improving test suite simplification performance.

All materials related to this study are publicly available at https://github.com/GLY-CETest/ChatTSS

2. BACKGROUND

2.1 Test Suite Simplification

TSR, sometimes referred to as test suite minimization [1], aims to eliminate redundant or obsolete test cases from the baseline test suite, thereby effectively reducing the cost of regression testing. However, this process is not without risks, as there is the potential to mistakenly remove test cases capable of detecting defects, which may compromise the fault detection ability of the test suite. Therefore, how to achieve a proper balance between reducing the number of test cases and maintaining fault detection capability has always been a critical focus in the research of test case reduction.

Following the research of Yoo et al. [1], Rothermel et al. [3], and Harrold et al. [22].

TSR can be formally defined as:

- ***Given*:** An original test suite, $T_o = \{t_1, t_2, ..., t_n\}$, a set of test requirements $R = \{r_1, ..., r_n\}$, that must be satisfied to provide the desired "adequate" testing of the program, and subsets of T_o, $T_1, ..., T_n$, one associated with each of the r_is such that any one of the test cases t_j belonging to T_i can be used to achieve requirement r_i.
- ***Problem*:** Find a representative set, $T_s = \{t_1, t_2, ..., t_m\}$, of test cases from T_o that satisfies R.

According to the above definition, TSR only reduces the size of the test suite during execution without modifying the individual test cases themselves. In contrast, CHATTSS decomposes test cases into finer-grained units called test atoms (as described in 2-B). Therefore, we refer to CHATTSS as a test case simplification approach, to distinguish it from conventional TSR.

2.2 Test Atoms

Unit test cases are generally expected to exhibit independence, repeatability, and maintainability, with a concise and clear structure to facilitate their reuse in subsequent regression testing. In the test suite simplification process, test cases are simplified based on specific requirements, which typically include coverage criteria. To further enhance the efficiency of TSR, we intuitively assume that reducing the coupling both between test cases and among multiple assertions within a single test case can help more precisely determine the test scope of each case and the mutants it can kill (if any). Motivated by this idea, we adopt a test case decomposition strategy that decomposes test cases into finer-grained "test atoms", each containing only a single assertion. This approach improves the independence and specificity of individual test units, contributing to more effective test suite simplification.

In the example shown in the following Fig.1(a), there are a total of two test assertions. In this example, the first assertion on line 7 is used to test the method for adding a user to confirm that the user can be successfully added. The second assertion on line 10 is to test the method for deleting a user, which depends on the user addition operation in the first test case. However, this dependency relationship is not obvious because, on the surface, the deletion operation seems to be independent, but in fact, it depends on the result of the addition operation. If the addition operation fails, then the deletion operation is very likely to fail, but this dependency relationship is hidden in the test logic.

There are often dependencies among code segments within a test case, particularly between multiple assertions. By performing in-depth analysis of the test case code to identify the specific code fragments each assertion depends on, we can effectively split the test case and extract what we refer to as "test atoms", as shown in Fig.1(b). This, in turn, helps ensure that each test atom remains syntactically correct and compilable to the greatest extent possible.

3. METHODOLOGY

The workflow of CHATTSS can be divided into three main components: data preprocessing, test decomposition, and test suite simplification, as illustrated in Fig.2. First, the test suite undergoes preprocessing operations such as cleaning and filtering to reduce the negative impact of low-quality data (i.e., test cases that do not conform to syntactic standards) on the

```java
@Test(timeout=4000)
void testAddAndDeleteUser() {
    UserRepository userRepository = new
        UserRepository();
    User user = new User("JohnDoe",
        "john.doe@example.com");
    userRepository.addUser(user);
    User retrievedUser =
        userRepository.getUserById(user.getId());
    assertNotNull(retrievedUser);
    userRepository.deleteUser(user.getId());
    retrievedUser =
        userRepository.getUserById(user.getId());
    assertNull(retrievedUser);



}
```

(a) An example of a test case containing multiple assertions to be decomposed.

```java
@Test(timeout=2000)
void testAddUser() {
    User user = new User("JohnDoe",
        "john.doe@example.com");
    userRepository.addUser(user);
    User retrievedUser =
        userRepository.getUserById(user.getId());
    assertNotNull(retrievedUser);}

@Test(timeout=2000)
void testDeleteUser() {
    User user = new User("JohnDoe",
        "john.doe@example.com");
    userRepository.addUser(user);
    userRepository.deleteUser(user.getId());
    User retrievedUser =
        userRepository.getUserById(user.getId());
    assertNull(retrievedUser);
}
```

(b)Two test atoms derived by decomposing one test case.

Figure 1: An example illustrating how a test case can be decomposed into multiple test atoms

subsequent TSR process. Next, each test case is decomposed into independent test atoms through code dependency analysis. Finally, based on the simplification requirements, a multi-objective optimization algorithm is applied to minimize the test atoms.

3.1 Preprocessing

The quality of test cases within a test suite is directly influenced by the proficiency of the testers in writing test code. When test cases are created collaboratively—i.e., involving multiple testers—the overall quality may become inconsistent. Low-quality data can negatively affect subsequent analysis and optimization processes. Moreover, executing redundant test cases may lead to unnecessary consumption of computational resources. Therefore, to minimize the potential adverse impact of low-quality data on the following steps, it is essential to perform data preprocessing as an initial stage.

Generally, the design of unit tests follows a set of common test case design guidelines, similar to coding standards, to ensure the quality of the test cases. These specifications cover multiple aspects, including but not limited to test naming specifications. A good test naming specification can make test cases more readable and maintainable, enabling developers and testers to understand the purpose and scope of the test quickly. In addition, the specification of the test code structure cannot be ignored. A reasonable test code structure helps to improve the execution efficiency and extensibility of tests and facilitates the modification and optimization of test cases in the future. There is also the use of assertions. Correct use of assertions can effectively verify whether the output of the program meets expectations and ensure the correctness of the program. At the same time, exception handling is also a crucial link. Appropriate exception handling can improve the stability and reliability of the program and can capture and handle exceptions in a reasonable manner in case of abnormal situations.

Based on the above design specifications, we have formulated the following preprocessing operations for crowdsourcing test cases:

1) **Delete failed compilation test cases (DFCTCs).** For test cases that fail to compile, issues typically arise in areas such as variable definitions, class instantiations, and method invocations.
2) **Delete non-assertion test cases except specific ones (RNATCES).** A unit test case must include at least one assertion to verify the actual output. Test cases without assertions not only fail to constitute a complete test but also compromise the accuracy of mutation analysis, as they affect the determination of whether a mutant is killed.
3) **Standardize test method and class names (STMNs).** Taking JUnit as an example, test methods should follow the naming convention "$test\{methodName\}$", and test classes should be named using the "$\{ClassName\}Test$" format. Such standardized naming conventions improve code readability. We first standardize the names of test methods and classes, and then refactor mixed test methods by distributing them into their corresponding test classes.
4) **Delete blank test classes (DBTCs).** After completing the above preprocessing work, if there are still blank test classes, we will delete them.

After the aforementioned preprocessing of test cases, the test code becomes more standardized, making it easier for the LLM to understand the code and perform subsequent operations.

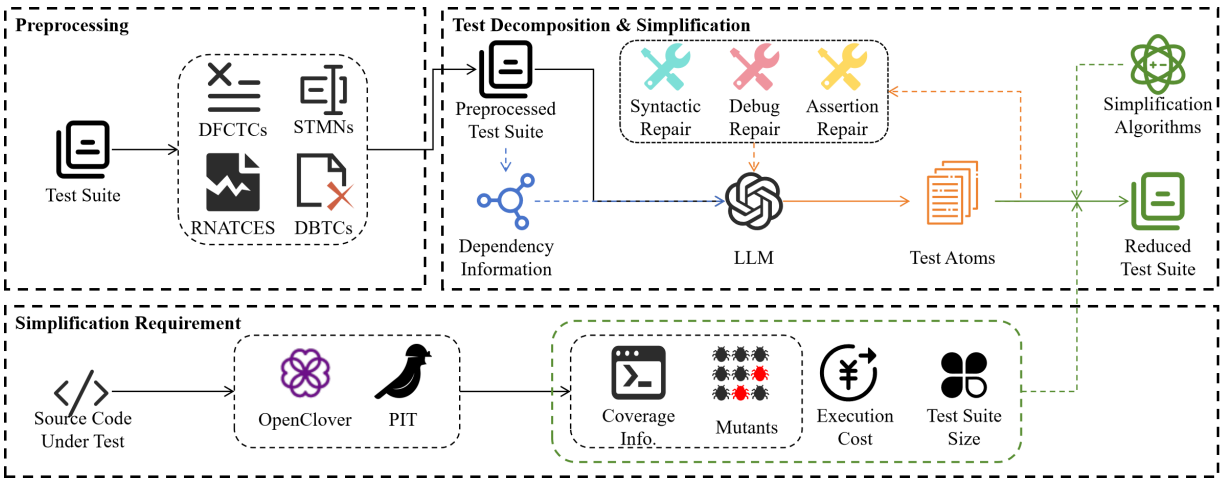

Figure 2: Workflow of CHATTSS

3.2 Simplification Requirements

In the following text, we use $T_o = \{t_1, t_2, ..., t_n\}$ to denote the original test suite composed of all original test cases, $T_d = \{t_1, t_2, ..., t_m\}$ to denote the test suite composed of all test atoms obtained in the previous test case decomposition, and $T_s = \{t_1, t_2, ..., t_l\}$ to denote the simplified test case suite composed of simplified test atoms, where m is less than or equal to n.

In the TSR process, reduction requirements are defined as specific criteria that the simplified test cases must satisfy [23]. These requirements may include the level of structural code coverage achieved, the number of mutants killed, or the functional points covered by the reduced test suite. Relevant studies have shown that adopting inadequate reduction requirements—i.e., allowing the reduced test suite T_s to fall short of fully satisfying all the criteria met by the original suite T_o, such as complete code coverage—is acceptable in practice. In fact, such relaxation often leads to improved reduction effectiveness in real-world scenarios [6][23][24][25]. Similarly, we adopt inadequate requirements in the simplification process of test atoms.

- **Branch Coverage.** Test atoms are simplified based on branch coverage of the source code. Specifically, if a test atom $t_i \in T_d\,(i \in [1, m])$ covers only source code branches that have already been covered by other test atoms, then t_i will be eliminated. We adopt an inadequate branch coverage simplification strategy, where T_s and T_o satisfy the following relationship:

$$|Cov\,(T_s) \cap Cov\,(T_o)| \leq |Cov\,(T_o)| \tag{1}$$

  where $Cov\,(T)$ denotes the code branches covered by the test suite T. The reason for using $|Cov\,(T_s) \cap Cov\,(T_o)|$ instead of $|Cov\,(T_s)|$ is that T_d and T_s may cover code branches that are not covered by T_o.
- **Mutation Score.** Similarly, based on mutants generated for the source code, test atoms are simplified through mutation testing. If a certain test atom $t_i \in T_d\,(i \in [1, m])$ does not kill more mutants than other test atoms, then t_i will be simplified. Eventually, T_s and T_o will satisfy the following relationship:

$$|Mu\,(T_s) \cap Mu\,(T_o)| \leq |Mu\,(T_o)| \tag{2}$$

  where $Mu\,(T)$ represents the mutants killed by the test suite T. The reason for using $|Mu\,(T_s) \cap Mu\,(T_o)|$ instead of $|Mu\,(T_s)|$ is that there may be differences between mutants generated each time.
- **Test Suite Size.** One of the primary goals of TSR is to obtain a smaller-sized test suite. While satisfying requirements such as coverage, minimizing the size of the test suite is also a critical TSR requirement.
- **Execution Cost.** Reducing execution cost is a crucial requirement in regression testing. While TSR focuses on minimizing the size of the test suite, it must also aim to reduce the overall execution cost as much as possible.

3.3 Test Decomposition & Simplification

During the development of test cases, in addition to adhering to basic coding standards, it is generally required that each test case can be executed independently. This requirement not only ensures the reliability of test results but also effectively prevents interference between test cases, thereby maintaining the integrity and accuracy of the testing process. To minimize the impact of test case dependencies on the effectiveness of simplification, CHATTSS performs in-depth analysis of test cases and, based on this analysis, decomposes them into finer-grained test atoms.

3.3.1 Dependency Analysis

During the design and implementation of test case code, in addition to the aforementioned coding standards, it is generally required that each test case be independently executable, with no ordering dependencies among test cases. This ensures

the reliability of test results and avoids interference between multiple test cases.

However, in practice, some test suites may contain inter-dependent test cases. To ensure the quality of the resulting test atoms after decomposition, it is necessary to perform dependency analysis on such cases. For this purpose, we employ abstract syntax tree (AST) and call graph (CG), which are commonly used in code dependency analysis, to identify and preserve the dependency chains among test cases. During the subsequent test case decomposition process, if a test case is found to be part of a dependency chain, we retrieve all related test cases using the recorded dependency information and perform decomposition on the entire group using the LLM.

3.3.2 Test Case Decomposition

As mentioned above, each test atom is indivisible and has the property of continuous execution to ensure the independence of the test process and the accuracy of the results. After obtaining the test atoms, in order to ensure the smooth progress of TSR, since it is difficult to ensure that all test atoms can be executed normally, we check the correctness of the test atoms and try to repair the test atoms that cannot be executed.

a) Prompt Design

After identifying the number of assertions in each test case, for test cases that need to be decomposed (i.e., when the number of assertions is more than one), we render them into a prompt template. Due to space limitations, the prompt template is not presented here, but we have made it available in our repository. The prompt contains each assertion and the line number of the code it depends on. At the same time, in order to maintain the consistency of the test environment and context, the LLM is required to reuse the initialization variables and related statements in the original test case and output the decomposed test cases according to the provided test case template.

b) Response Processing

During the attempt, we realize that we could not directly convert the response of the LLM into JUnit4 code and save it to a code file. The reason is that the response of the LLM sometimes contains "```java", or the code is incomplete due to token limitations.

Therefore, in this process, we first use regular expressions to filter out "```java" and "```" that may exist in the code of the LLM response. Then, we check the integrity of the code by checking the number of curly braces in the code. If the number of right curly braces is less than that of left curly braces, we complete the curly braces. In addition, if a code segment containing "@Test" does not contain an assertion, it will be directly deleted, and then the curly braces in the remaining code will be completed.

c) Test Atoms Repair

After undergoing the above processing, the test cases are transformed into test atoms. In theory, these test atoms can be executed independently without any execution order dependencies. However, this does not guarantee that all test atoms will run correctly. One reason is that the test atom files may lack the necessary package declarations or import statements for required dependencies. Additionally, since multiple test atoms may originate from the same test case, there is a high likelihood of duplicate method names. Furthermore, errors introduced during the decomposition process may cause some test atoms to fail during execution.

To address the first issue, we supplement the test atoms with appropriate package declarations and import statements based on syntactic rules. For test atoms with duplicate method names, we append unique identifiers to avoid naming conflicts. In the case of execution failures, we leverage test execution feedback and the capabilities of LLM for automated repair. These failures generally fall into two categories: the first involves syntax or runtime errors that prevent the test atom from successfully executing on mutated versions; the second involves assertion failures—despite providing code dependency information during decomposition, it is not always possible to guarantee that the transformation preserves the original testing logic, potentially leading to previously passing assertions becoming failing ones. We execute the test atoms and capture the resulting error messages, analyze the root causes of the failures, and feed this information to the LLM to repair the faulty test atoms. If the LLM fails to repair a test atom after three attempts or if the token limit is exceeded, the test atom is discarded.

3.3.3 Test Suite Simplification

Essentially, generating a test case set after has as its core objective obtaining a set that can achieve minimum coverage. In the field of computer science, this problem is called the NP-complete problem [4][26]. At the same time, it can definitely be regarded as a multi-objective optimization problem. This clearly means that when we face the arduous task of generating a simplified test case set, finding an optimal solution is extremely challenging. Because this process is very likely to require a large amount of computing resources and a long time.

In this paper, we fully consider taking branch coverage and mutation test scores as the key requirements for reducing test cases and using multi-objective optimization algorithms to perform simplification in order to improve the efficiency and quality of the test case set to a certain extent.

Search-based algorithms are commonly applied in TSR, with the greedy algorithm being the most widely used. Although various variants of the greedy algorithm have emerged to address multi-objective optimization problems, along with the development of more complex and powerful multi-objective optimization algorithms, the greedy algorithm still retains certain advantages in TSR under specific circumstances.

It must be acknowledged that no single optimization algorithm can effectively handle all scenarios. To evaluate the impact of different optimization algorithms on the simplification performance of CHATTSS, we applied four algorithms: greedy algorithm (MOGA), non-dominated sorting genetic algorithm II (NSGA-II) [27], strength pareto evolutionary algorithm

2 (SPEA2) [28], and multi-objective evolutionary algorithm based on decomposition (MOEA/D) [29].

4. EXPERIMENT

4.1 Research Questions

To evaluate CHATTSS, the following research questions are designed to guide the experiments:

***RQ1*: Does the LLM possess sufficient program analysis capability to accomplish the task of test case decomposition?**

***RQ2*: What is the effectiveness of CHATTSS in TSR?**

***RQ3*: What is the impact of optimization algorithm selection on CHATTSS when dealing with different optimization objectives?**

For RQ1, the role of the LLM in the process of test case decomposition is evaluated by comparing the quality before and after the decomposition. The LLM must correctly understand the program and analyze the dependencies within the test code to accurately accomplish the test case decomposition task.

For RQ2, the study investigates the efficiency of CHATTSS. The impact of test case decomposition on simplification will also be analyzed by comparing different levels of simplification (i.e., test suite, test class, test method, and test atom).

The goal of RQ3 is to investigate how different optimization algorithms affect the efficiency of test suite simplification in CHATTSS when faced with varying simplification requirements (i.e., different optimization objectives). This analysis will help us understand how to select the most suitable optimization algorithm based on the specific characteristics and simplification requirements of different testing projects.

4.2 Subjects

To achieve a more comprehensive evaluation, this study utilizes test cases from both real-world testing projects and benchmark datasets. A portion of the data was obtained from the MoocTest [1], which is one of the largest crowdsourced testing platforms in China. On this platform, users can upload testing targets and set reward amounts to publish testing tasks. Representative projects from MoocTest include BPlusTree, Chef, and EightQueens. Another set of projects was selected from the Defects4J dataset, including Jsoup, Gson, Commons-Lang, and Math. All testing tasks were built using Maven [2] and executed with the JUnit4 [3] testing framework.

Totally, we obtain over 10,000 test suites from seven different testing tasks. All the test cases are publicly available, and the detailed information of each testing task is presented in TableI. Column 1 lists the names of the projects under test; column 2 shows the number of lines of code (#LOC) in the projects. Column 3 indicates the number of test cases (#TCs) collected for each project. To control the randomness of mutant generation, we select several mutators provided by PIT that exhibit relatively low randomness, including INCREMENTS, RETURN_VALS, INVERT_NEGS, CONGDITIONS_BOUNDARY, MATH, VOID_METHOD_CALLS, and NEGATE_CONDITIONALS.

TABLE I: Statistics about the projects used in our evaluation.

Project	#LOC	#TCs
BPlusTree	474	328
Chef	716	727
EightQueens	295	541
Jsoup	2,546	139
Gson	7,857	1,029
Commons-Lang	21,785	2,259
Math	84,615	5,246

However, not all test cases are executable or meet evaluation requirements. To further filter the test suites, this study first locally builds each test suite and its associated program, excluding those that lead to build failures. Next, test cases are evaluated for the presence of JUnit assertion statements; if none are found, the test case is considered low-quality and is removed.

4.3 Experiment Settings

In this experiment, we employ the non-adequate TSR. Based on the findings of Shi et al. [8], during the simplification process, we drop down the simplification requirement to 95%. This means the simplified test suite only needs to meet at least 95% of the requirements of the original test suite. Specifically, the simplified test suite must cover no less than 95% of the code branches covered by the original test suite, and it must kill no less than 95% of the mutants killed by the original test suite.

We conduct the experiment on a PC running Windows 11, equipped with an 11th Gen Intel(R) Core(TM) i7-11800H @ 2.30GHz CPU and 32GB of RAM. The LLM we use is GPT-3.5-turbo.

4.4 Results and Analysis

4.4.1 RQ1: The Program Analysis Capability of CHATTSS.

a) Experimental Design

To evaluate the test case decomposition capability of CHATTSS, we performed the test case decomposition task on the test suites of seven projects. For each project, we recorded the following metrics: the number of test case (#TCs) in the original test suite (T_o), the number of test atoms generated after decomposition (#TAs), the initial pass rate of the test atoms (IPR), and the pass rate after repair (RPR).

b) Result and Analysis

We compared the number of test cases (#TCs) and test atoms (#TAs) before and after test case decomposition across different projects. The experimental data is presented in column 1

[1]http://www.mooctest.net/

[2]https://maven.apache.org/

[3]https://junit.org/junit4/

TABLE II: Comparison between the number of original test cases and test atoms, as well as the initial pass rate of test atoms after the first decomposition and the pass rate after up to three rounds of repair.

Project	#TCs	#TAs	IPR(%)	RPR(%)
BPlusTree	328	688	0.85	0.94
Chef	727	999	0.86	0.95
EightQueens	541	884	0.90	0.98
Jsoup	139	346	0.94	1.00
Gson	1,029	1,553	0.86	0.96
Commons-Lang	2,259	7,891	0.81	0.98
Math	5,246	9,150	0.80	0.93

and 2 of Tab.II. The number of test atoms increased to varying degrees compared to the number of original test cases, with Commons-Lang exhibiting a threefold increase.

Columns 3 and 4 of Tab.II summarize the test atom repair results. Among the initially generated test atoms, a certain proportion failed due to syntax or assertion errors. Notably, the Math project had the lowest initial test atom pass rate, at only 80%. However, after the repair process, the pass rate of test atoms in all projects exceeded 93%, demonstrating CHATTSS's strong capability in repairing and refining test atoms.

Answer to RQ1

The analysis of the results indicates that the number of test atoms increased significantly compared to the original test cases, and the extent of this increase varied depending on the quality of the test cases in each project. CHATTSS demonstrated a strong capability in repairing test atoms, with the post-repair pass rate exceeding 93% across all evaluated projects.

4.4.2 RQ2: The Impact of Test Atoms on TSR and the Simplification Performance of CHATTSS.

*a) **Experimental Design.***

To empirically assess the effectiveness of test atoms in enhancing the efficiency of Test Suite Simplification, we conducted a series of experiments. Test atoms, by nature, are a finer-grained abstraction of test cases. By applying TSR with different targets—test atoms, test methods (i.e., original test cases), and test classes—we can analyze and compare the reduction rates, thereby offering an intuitive understanding of the influence of test atoms on TSR outcomes. We perform reduction tasks with the minimum units being test classes (TCR), test methods (TMR), and test atoms (TAR), respectively. By referencing relevant studies [1][8][23], we compare the three reduction methods using the following metrics:

- **Reduction Ratio (RR).** RR represents the scale change of T_s compared with T_o after TSR and is defined as:

$$RR = \frac{|T_o| - |T_s|}{|T_o|} \times 100\%$$

where $|T_o|$ represents the number of test cases in T_o, and $|T_s|$ represents the number of test cases in T_s.

- **Branch Coverage Loss (BCL).** BCL represents the loss of branch coverage achieved by T_s compared with T_o after TSR and is defined as:

$$BCL = \frac{|Cov_b(T_o)| - |Cov_b(T_s)|}{|Cov_b(T_o)|} \times 100\%$$

where $|Cov_b(T_o)|$ represents the branches covered by T_o, and $|Cov_b(T_s)|$ represents the branches covered by T_s.

- **Mutant Detection Loss (MDL).** MDL represents the loss of mutants killed by T_s compared with T_o after TSR and is defined as:

$$MDL = \frac{|Mut(T_o)| - |Mut(T_s)|}{|Mut(T_o)|} \times 100\%$$

where $|Mut(T_o)|$ represents the number of mutants killed by T_o, and $|Mut(T_s)|$ represents the number of mutants killed by T_s.

*b) **Result and Analysis.***

To evaluate the impact of test atoms, we employed the relatively simple MOIGA algorithm as the reduction algorithm in this experiment. Due to the inherent randomness of optimization algorithms, we conducted 10 repeated experiments, and the final average results were recorded as shown in Tab.III. In the seven projects, all three methods of reduction using different reduction objects achieved zero loss in branch coverage and fault detection rate (as shown in columns 4-9 of Tab.III). In terms of reduction rate, it was clearly observed that the reduction rate using test atoms was higher than the other two methods for each project (as shown in columns 1-3 of Tab.III). Among them, CHATTSS achieved the highest reduction rate in the BPlusTree project, with a rate of 97.58%. We also observed that the reduction rates of the first three projects were significantly higher than those of the latter four, suggesting that the test data quality in the Defects4J dataset is relatively higher. Specifically, in terms of coverage and fault detection capability, it contains fewer redundant tests.

Answer to RQ2

The reduction efficiency of CHATTSS has been validated, with its reduction rate being the highest across all projects. This also demonstrates that test atoms play a significant and positive role in the TSR process. These findings further validate the effectiveness of our proposed test suite simplification approach, highlighting its advantages over conventional test suite reduction.

TABLE III: The reduction statistics for CHATTSS. TCR, TMR, and TAR represent the reductions performed with the test class, test method, and test atom as the minimum units.

Project	RR(%)			BCL(%)			FDL(%)		
	TAR	TMR	TCR	TAR	TMR	TCR	TAR	TMR	TCR
BPlusTree	**97.58**	88.40	71.38	0.00	0.00	0.00	0.00	0.00	0.00
Chef	**97.53**	93.95	81.06	0.00	0.00	0.00	0.00	0.00	0.00
EightQueens	**95.38**	83.99	53.55	0.00	0.00	0.00	0.00	0.00	0.00
Jsoup	**60.98**	31.65	1.44	0.00	0.00	0.00	0.00	0.00	0.00
Gson	**65.68**	55.98	4.37	0.00	0.00	0.00	0.00	0.00	0.00
Commons-Lang	**67.35**	37.67	7.25	0.00	0.00	0.00	0.00	0.00	0.00
Math	**67.35**	37.67	7.25	0.00	0.00	0.00	0.00	0.00	0.00

4.4.3 RQ3: The Impact of Simplification Algorithms on CHATTSS.

*a) **Experimental Design.***

To explore the impact of different simplification algorithms on CHATTSS, we incorporate multiple objectives into the simplification process through multi-objective optimization. In practical applications, flexibly selecting appropriate multi-objective optimization algorithms based on different scenarios may yield better results.

To construct diverse optimization scenarios, we adopt the simplification objectives described in 3-C3 and design two types of simplification tasks: a three-objective optimization task and a four-objective optimization task. The corresponding optimization objectives are defined as follows:

- **Three-objective:** branch coverage (BC), fault detection rate (FDR), and execution cost (EC).
- **Four-objective:** branch coverage, fault detection rate, execution cost, and test suite size (evaluated with simplification rate).

In addition to the simplification objectives, we also employed Hypervolume (HV)—a commonly used metric in multi-objective optimization—to evaluate the results of TSR. The HV metric provides a quantitative measure of the quality of the obtained Pareto front by computing the volume of the objective space dominated by the solution set. The calculation formula for HV is shown below:

$$HV = \delta(\bigcup_{i=1}^{|s|} v_i)$$

where δ denotes the Lebesgue measure, which is used to quantify the volume. $|S|$ represents the number of non-dominated solutions in the solution set, and v_i refers to the hypervolume formed between the reference point and the i-th solution in the set. A larger HV value indicates higher solution quality and a better distribution along the Pareto front.

All optimization algorithms in the experiments were configured with identical parameter settings: a population size of 100, 4000 generations, a crossover probability of 1.0, and a mutation probability of $(1/n)$, where n is the size of the test suite. All experiments were conducted under the same computational environment.

*b) **Result and Analysis.***

Tab.IV and Tab.V present the performance of CHATTSS when employing different simplification algorithms to solve the three-objective and four-objective optimization tasks, respectively. The algorithm with the highest HV score in each project is highlighted in bold to indicate superior performance.

In the **three-objective** optimization task, the experimental results show that NSGA-II consistently achieved the best HV values across multiple projects. Specifically, NSGA-II obtained the highest HV scores on the BPlusTree, Chef, Jsoup, and Commons-Lang projects, reaching 0.9094, 0.6163, 0.5714, and 0.6657, respectively. These results indicate that the test suites optimized by NSGA-II achieved a well-balanced performance in terms of coverage, fault detection capability, and execution cost. Although MOIGA performed well on some small-scale projects (e.g., EightQueens), its performance on large-scale testing tasks (such as Math and Commons-Lang) was less efficient, with execution costs (EC) significantly higher than those of the other algorithms.

In the **four-objective** optimization task, the MOIGA demonstrated relatively high HV values on small-scale projects such as BPlusTree and EightQueens, achieving 0.7829 and 0.7816, respectively. These results indicate that the algorithm outperformed others in test suite simplification effectiveness while maintaining the lowest execution cost. This advantage can be attributed to its strategy of selecting, at each iteration, the test case with the highest contribution to the optimization objectives, enabling the rapid identification of high-impact test cases. However, the performance of the incremental greedy algorithm degraded significantly on large-scale testing tasks, such as Math and Commons-Lang, with HV values of only 0.1078 and 0.2641, respectively. This suggests that the resulting test suites were suboptimal in terms of fault detection rate (FDR), and execution cost (EC). The primary reason lies in the heuristic nature of the algorithm, which tends to fall into local optima during complex optimization processes.

In contrast, NSGA-II achieved superior HV results across

TABLE IV: Performance of three-objective simplification algorithms

Project	Algorithm	BC(%)	FDR(%)	EC(ms)	HV
BPlusTree	NSGA-II	100	91	160	**0.9094**
	SPEA2	100	89	160	0.8867
	MOEA/D	100	89	420	0.8812
	MOIGA	100	91	170	0.9052
Chef	NSGA-II	91	67	200	**0.6163**
	SPEA2	91	66	200	0.6000
	MOEA/D	91	64	310	0.5118
	MOIGA	91	70	300	0.5472
EightQueens	NSGA-II	100	85	90	0.8442
	SPEA2	100	85	240	0.8270
	MOEA/D	100	84	480	0.7973
	MOIGA	100	85	60	**0.8443**
Gson	NSGA-II	91	66	1200	**0.5633**
	SPEA2	91	67	1220	0.5623
	MOEA/D	90	61	2130	0.4253
	MOIGA	92	76	2410	0.5147
Jsoup	NSGA-II	90	63	230	**0.5714**
	SPEA2	90	62	180	0.5648
	MOEA/D	89	61	270	0.5354
	MOIGA	90	65	760	0.5250
Commons-Lang	NSGA-II	97	69	3690	**0.6657**
	SPEA2	97	67	3710	0.6652
	MOEA/D	93	55	7470	0.4949
	MOIGA	98	84	82020	0.3918
Math	NSGA-II	91	56	48106	**0.5007**
	SPEA2	92	55	60799	0.4912
	MOEA/D	88	45	659990	0.3790
	MOIGA	92	66	7668000	0.1968

TABLE V: Performance of four-objective simplification algorithms

Project	Algorithm	BC(%)	FDR(%)	RR(%)	EC(ms)	HV
BPlusTree	NSGA-II	100	89	83	190	0.7635
	SPEA2	100	90	84	170	0.7748
	MOEA/D	100	84	64	580	0.5323
	MOIGA	100	91	86	170	**0.7829**
Chef	NSGA-II	91	65	87	140	**0.5819**
	SPEA2	91	64	85	130	0.5344
	MOEA/D	91	64	65	230	0.3502
	MOIGA	91	70	92	300	0.5074
EightQueens	NSGA-II	100	84	87	90	0.7448
	SPEA2	100	85	89	80	0.7663
	MOEA/D	100	76	66	820	0.4641
	MOIGA	100	85	92	60	**0.7816**
Gson	NSGA-II	91	61	69	923	**0.4106**
	SPEA2	91	59	73	949	0.3638
	MOEA/D	89	56	59	1720	0.2479
	MOIGA	92	76	64	2410	0.3339
Jsoup	NSGA-II	89	57	73	370	**0.4612**
	SPEA2	89	44	81	130	0.3902
	MOEA/D	88	52	67	280	0.3244
	MOIGA	90	65	60	760	0.3190
Commons-Lang	NSGA-II	96	60	60	4640	**0.3756**
	SPEA2	96	59	62	4400	0.3499
	MOEA/D	93	54	52	6060	0.2562
	MOIGA	98	84	67	82020	0.2641
Math	NSGA-II	89	44	60	551400	**0.2722**
	SPEA2	89	44	62	704210	0.2461
	MOEA/D	87	42	53	687670	0.1879
	MOIGA	92	66	54	7668000	0.1078

most projects. For instance, it obtained the highest HV values on the Gson, Jsoup, and Math projects, reaching 0.4106, 0.4612, and 0.2722, respectively. These results suggest that NSGA-II produced test suites with better overall balance across multiple objectives. Moreover, on the large-scale Math project, NSGA-II significantly outperformed the MOIGA in terms of HV (0.2722 vs. 0.1078), demonstrating its stronger adaptability and robustness in handling complex multi-objective optimization tasks.

Answer to RQ3

Due to its use of non-dominated sorting and a crowding distance mechanism, NSGA-II enables CHATTSS to fully leverage its strengths in large-scale testing tasks. In particular, under large-scale software testing scenarios, NSGA-II can significantly reduce test execution costs while maintaining high testing quality, thereby improving overall testing efficiency.

5. THREATS TO VALIDITY

Although we have made every effort throughout the experimental process to minimize factors that may threaten the validity of our results, it is undeniable that certain threats remain difficult to eliminate entirely.

With respect to internal validity, the primary threat arises from the characteristics of the benchmark projects we selected. During the selection process, we primarily relied on metrics such as lines of code and code complexity to estimate the difficulty of each project. However, this approach has certain limitations. It may result in the inclusion of projects that appear relatively simple on the surface but are, in fact, more challenging to test in practice. For example, one of the projects contains a large number of private methods, which are inherently difficult for both testers and LLM to handle effectively, as it is often unclear how such methods should be tested. From another perspective, since the selected projects are all carefully curated open-source repositories from GitHub, it is possible that some online LLM may have encountered similar codebases during their pre-training phase. This could potentially lead to overly optimistic experimental results, as the models may perform better than they would on truly unseen and novel code.

The external threats to our experiment mainly arise from the fact that we only consider Java-related projects. For projects written in other programming languages, the inherent differences between languages and the varying levels of LLM proficiency in handling different programming languages could lead to less optimal results when applying CHATTSS to non-Java projects.

6. RELATED WORK

TSR typically employs metrics such as code coverage (e.g., statement coverage [6][9][10][11], block coverage [30][31], branch coverage [32][33][4], modified condition/decision coverage (MC/DC coverage) [4][15], fault detection [10][13][14], and execution cost (computational cost or execution time) [4][15][34] as the requirements for TSR.

In subsequent related studies, researchers find that by appropriately relaxing the constraints on test requirements—meaning that the simplified test suite (T_s) no longer fully satisfies all the original test requirements (R)—the efficiency of TSR could be significantly improved. This approach, which allows for a controlled loss in fulfilling test requirements within acceptable limits, is referred to as *inadequate reduction*. Alipour et al. propose a similar reduction method, referred to as *non-adequate test reduction* [23].

Depending on the specific reduction requirements, TSR is treated as different optimization problems, generally classified into single-objective optimization [16][31][35][36][37][38] and multi-objective optimization[4][39][40][41]. In multi-objective optimization, to address various complex simplification requirements, a variety of optimization algorithms are employed. The algorithms used in TSR can be broadly categorized into greedy algorithms, clustering algorithms, meta-heuristic algorithms, search algorithms, and hybrid algorithms. Among these, greedy algorithms and search algorithms are the most widely applied.

Hybrid reduction algorithms are categorized into Intra-hybrid and Inter-hybrid based on the type of algorithms selected [18]. The general approach of hybrid algorithms is to divide the TSR process into several steps and select appropriate algorithms for each step based on the specific characteristics of that step [16][42].

7. CONCLUSION AND FUTURE WORK

In this study, we proposed CHATTSS, a test suite simplification approach based on LLM. Leveraging the program analysis capabilities of LLM, CHATTSS extracts code dependencies within test cases and decomposes them into fine-grained test atoms. These atoms are then simplified using multi-objective optimization algorithms to generate a more efficient test suite. To demonstrate the effectiveness of CHATTSS in terms of test case decomposition and simplification efficiency, we conducted an empirical evaluation on seven projects comprising over 10,000 test cases. The results confirm that CHATTSS exhibits superior performance in TSR.

Given these promising results, we plan to further improve the proposed approach as part of our future work. In our current experiments, we compared four different simplification algorithms. Moving forward, we aim to enhance CHATTSS with the ability to automatically select the most suitable optimization algorithm based on the characteristics of the testing project. This will help maximize the quality of the simplified test suite while minimizing the computational cost of CHATTSS.

ACKNOWLEDGEMENTS

The authors would like to thank the anonymous reviewers for their insightful comments. This work is supported partially by Shenzhen-Hong Kong-Macau Science and Technology Program(SGDX20230821091559018).

REFERENCES

[1] S. Yoo and M. Harman, "Regression testing minimization, selection and prioritization: a survey," *Software testing, verification and reliability*, vol. 22, no. 2, pp. 67–120, 2012.

[2] L. Zhang, D. Marinov, L. Zhang, and S. Khurshid, "An empirical study of junit test-suite reduction," in *2011 IEEE 22nd International Symposium on Software Reliability Engineering*, pp. 170–179, IEEE, 2011.

[3] G. Rothermel, M. J. Harrold, J. Von Ronne, and C. Hong, "Empirical studies of test-suite reduction," *Software Testing, Verification and Reliability*, vol. 12, no. 4, pp. 219–249, 2002.

[4] W. Zheng, R. M. Hierons, M. Li, X. Liu, and V. Vinciotti, "Multi-objective optimisation for regression testing," *Information Sciences*, vol. 334, pp. 1–16, 2016.

[5] L. S. de Souza, P. B. de Miranda, R. B. Prudencio, and F. d. A. Barros, "A multi-objective particle swarm optimization for test case selection based on functional requirements coverage and execution effort," in *2011 IEEE 23rd International Conference on Tools with Artificial Intelligence*, pp. 245–252, IEEE, 2011.

[6] S. Yoo and M. Harman, "Pareto efficient multi-objective test case selection," in *Proceedings of the 2007 international symposium on Software testing and analysis*, pp. 140–150, 2007.

[7] A. Marchetto, G. Scanniello, and A. Susi, "Combining code and requirements coverage with execution cost for test suite reduction," *IEEE Transactions on Software Engineering*, vol. 45, no. 4, pp. 363–390, 2017.

[8] A. Shi, A. Gyori, M. Gligoric, A. Zaytsev, and D. Marinov, "Balancing trade-offs in test-suite reduction," in *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*, pp. 246–256, 2014.

[9] J. Black, E. Melachrinoudis, and D. Kaeli, "Bi-criteria models for all-uses test suite reduction," in *Proceedings. 26th International Conference on Software Engineering*, pp. 106–115, IEEE, 2004.

[10] D. Hao, L. Zhang, X. Wu, H. Mei, and G. Rothermel, "On-demand test suite reduction," in *2012 34th International Conference on Software Engineering (ICSE)*, pp. 738–748, IEEE, 2012.

[11] C. Coviello, S. Romano, G. Scanniello, and G. Antoniol, "Gasser: Genetic algorithm for test suite reduction," in *Proceedings of the 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pp. 1–6, 2020.

[12] S. Tallam and N. Gupta, "A concept analysis inspired greedy algorithm for test suite minimization," *ACM SIGSOFT Software Engineering Notes*, vol. 31, no. 1, pp. 35–42, 2005.

[13] N. Gupta, A. Sharma, and M. K. Pachariya, "Multi-objective test suite optimization for detection and localization of software faults," *Journal of King Saud University-Computer and Information Sciences*, vol. 34, no. 6, pp. 2897–2909, 2022.

[14] A. Shi, A. Gyori, S. Mahmood, P. Zhao, and D. Marinov, "Evaluating test-suite reduction in real software evolution," in *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 84–94, 2018.

[15] W. Zheng, X. Wu, S. Cao, and J. Lin, "Ms-guided many-objective evolutionary optimisation for test suite minimisation," *IET Software*, vol. 12, no. 6, pp. 547–554, 2018.

[16] Z. Anwar, H. Afzal, N. Bibi, H. Abbas, A. Mohsin, and O. Arif, "A hybrid-adaptive neuro-fuzzy inference system for multi-objective regression test suites optimization," *Neural Computing and Applications*, vol. 31, pp. 7287–7301, 2019.

[17] S. Yoo and M. Harman, "Using hybrid algorithm for pareto efficient multi-objective test suite minimisation," *Journal of Systems and Software*, vol. 83, no. 4, pp. 689–701, 2010.

[18] A. S. Habib, S. U. R. Khan, and E. A. Felix, "A systematic review on search-based test suite reduction: State-of-the-art, taxonomy, and future directions," *IET Software*, vol. 17, no. 2, pp. 93–136, 2023.

[19] J. Bell, G. Kaiser, E. Melski, and M. Dattatreya, "Efficient dependency detection for safe java test acceleration," in *Proceedings of the 2015 10th joint meeting on foundations of software engineering*, pp. 770–781, 2015.

[20] A. Gyori, A. Shi, F. Hariri, and D. Marinov, "Reliable testing: Detecting state-polluting tests to prevent test dependency," in *Proceedings of the 2015 international symposium on software testing and analysis*, pp. 223–233, 2015.

[21] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*, pp. 643–653, 2014.

[22] M. J. Harrold, R. Gupta, and M. L. Soffa, "A methodology for controlling the size of a test suite," *ACM transactions on software engineering and methodology (TOSEM)*, vol. 2, no. 3, pp. 270–285, 1993.

[23] M. A. Alipour, A. Shi, R. Gopinath, D. Marinov, and A. Groce, "Evaluating non-adequate test-case reduction," in *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, pp. 16–26, 2016.

[24] A. Groce, M. A. Alipour, C. Zhang, Y. Chen, and J. Regehr, "Cause reduction for quick testing," in *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation*, pp. 243–252, IEEE, 2014.

[25] A. Groce, M. A. Alipour, C. Zhang, Y. Chen, and J. Regehr, "Cause reduction: delta debugging, even without bugs," *Software Testing, Verification and Reliability*, vol. 26, no. 1, pp. 40–68, 2016.

[26] G. M. Kapfhammer, "Regression testing.," 2010.

[27] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: Nsga-ii," *IEEE transactions on evolutionary computation*, vol. 6, no. 2, pp. 182–197, 2002.

[28] E. Zitzler, M. Laumanns, and L. Thiele, "Spea2: Improving the strength pareto evolutionary algorithm," *TIK report*, vol. 103, 2001.

[29] Q. Zhang and H. Li, "Moea/d: A multiobjective evolutionary algorithm based on decomposition," *IEEE Transactions on evolutionary computation*, vol. 11, no. 6, pp. 712–731, 2007.

[30] W. E. Wong, J. R. Horgan, S. London, and A. P. Mathur, "Effect of test set minimization on fault detection effectiveness," in *Proceedings of the 17th international conference on Software engineering*, pp. 41–50, 1995.

[31] S. Nachiyappan, A. Vimaladevi, and C. SelvaLakshmi, "An evolutionary algorithm for regression test suite reduction," in *2010 International Conference on Communication and Computational Intelligence (INCOCCI)*, pp. 503–508, IEEE, 2010.

[32] D. Jeffrey and N. Gupta, "Improving fault detection capability by selectively retaining test cases during test suite reduction," *IEEE Transactions on software Engineering*, vol. 33, no. 2, pp. 108–123, 2007.

[33] A. Bergel and V. Peña, "Increasing test coverage with hapao," *Science of Computer Programming*, vol. 79, pp. 86–100, 2014.

[34] Y. Liu, Z. Li, R. Zhao, and P. Gong, "An optimal mutation execution strategy for cost reduction of mutation-based fault localization," *Information Sciences*, vol. 422, pp. 572–596, 2018.

[35] C. Coviello, S. Romano, G. Scanniello, A. Marchetto, A. Corazza, and G. Antoniol, "Adequate vs. inadequate test suite reduction approaches," *Information and Software Technology*, vol. 119, p. 106224, 2020.

[36] M. Asthana, K. D. Gupta, and A. Kumar, "Test suite optimization using lion search algorithm," in *Ambient Communications and Computer Systems: RACCCS 2019*, pp. 77–90, Springer, 2020.

[37] K. Z. Zamli, N. Safieny, and F. Din, "Hybrid test redundancy reduction strategy based on global neighborhood algorithm and simulated annealing," in *Proceedings of the 2018 7th International Conference on Software and Computer Applications*, pp. 87–91, 2018.

[38] M. Khari and P. Kumar, "An effective meta-heuristic cuckoo search algorithm for test suite optimization," *Informatica*, vol. 41, no. 3, 2017.

[39] A. Choudhary, A. P. Agrawal, and A. Kaur, "An effective approach for regression test case selection using pareto based multi-objective harmony search," in *Proceedings of the 11th International Workshop on Search-Based Software Testing*, pp. 13–20, 2018.

[40] M. Bharathi and V. Sangeetha, "Weighted rank ant colony metaheuristics optimization based test suite reduction in combinatorial testing for improving software quality," in *2018 Second International Conference on Intelligent Computing and Control Systems (ICICCS)*, pp. 525–534, IEEE, 2018.

[41] A. Bajaj, O. P. Sangwan, and A. Abraham, "Improved novel bat algorithm for test case prioritization and minimization," *Soft Computing*, vol. 26, no. 22, pp. 12393–12419, 2022.

[42] L. Yan, W. Hu, and L. Han, "Optimize spl test cases with adaptive simulated annealing genetic algorithm," in *Proceedings of the ACM Turing Celebration Conference-China*, pp. 1–7, 2019.